

Dissection of XR Physics Engine into a Cloud-native Service

A.Protopsaltis^{3,5} , G. Kokiadis^{2,5} , M. Pateraki^{1,4,5} , M. Kamarianakis^{1,2,5} , M. Morfiadakis^{2,5}  & G. Papagiannakis^{1,2,5} 

¹Foundation of Technology Hellas - ICS, Heraklion, Greece

²University of Crete, Heraklion, Greece

³University of Western Macedonia, Kozani, Greece

⁴National Technical University of Athens, Athens, Greece

⁵ORamaVR S.A., Heraklion, Greece

Abstract

This paper addresses the demand for realistic physics simulations in collaborative Extended Reality (XR) experiences, particularly in gamified applications for untethered Head-Mounted Displays (HMDs). We propose optimizing the physics computations workload of modern game engines, by offloading computations to edge/cloud resources, serving multiple XR users in a 1-N fashion. Performance experiments with 25 users show improved rendering times under optimal network conditions, establishing thresholds for network latency and packet loss to maintain Quality of Experience (QoE) in XR applications. This approach enhances HMD battery life and user mobility, contributing to the advancement of realism and interactivity in XR environments.

CCS Concepts

• **Human-centered computing** → **Virtual reality**; Collaborative interaction; • **Software and its engineering** → Client-server architectures;

1. Introduction

Immersive collaborative XR experiences demand realistic simulations for optimal Quality of Experience (QoE) [PP20]. In mobile-XR environments, efficient physics algorithms are essential for handling 3D object animations, transformations, and soft-body deformations while maintaining interactivity. High-resolution, low-latency graphics rendering at a minimum frame rate of 60fps is crucial. Although modern game engines (MGEs) tackle many physics simulation challenges effectively, specific material physics aspects, such as liquid or deformable surfaces in GameObjects (GOs), are often overlooked, leading to unsatisfactory rendering. Standalone XR headsets with limited processing power often simplify physics models, affecting behavior alignment with expected norms. Tethering to high-end workstations improves processing power but limits user mobility, disrupting immersion.

In pursuit of advancing the efficiency and adaptability of MGEs for untethered HMDs in immersive XR environments, this paper introduces a novel approach to dissect the native Physics simulation engine from the main application. The primary goals of this Dissected Physics module are to minimize rendering time by alleviating the resource usage on untethered HMDs. Additionally, the module aims to provide an easily extendable and customizable Physics Engine capable of supporting and enhancing multi-player sessions without imposing user limitations. Furthermore, it aspires to maintain a headless persistent simulation state independent of users online. To achieve these goals, the module imposes constraints such



Figure 1: 2 HMD Users and the PhyS during a medical simulation.

as low latency on average networking conditions, optimal QoE for real-time interactive applications, the ability to handle numerous multi-data inter-calls from/to the Physics Engine, and compatibility with existing MGEs. Through this approach, we seek to create a seamless and immersive XR gaming experience while addressing the challenges posed by the limitations of untethered HMDs.

2. Related Work

MGEs offer a user-friendly interface, compatibility with major platforms, and support for various asset formats. Leveraging object-oriented design and a component-based architecture, MGEs utilize a built-in physics engine for realistic interactions between game objects. Online resources cover fundamental principles and usage of MGE's physics engine. However, the

longstanding issue of separating physics and rendering layers becomes significant in cloud/edge computing frameworks. Micro-services, popular in game development, have been explored, particularly in mobile cloud integration and back-end architectures for MMORPGs [Liu18]. The transitioning of MGE processes to cloud/edge computing necessitates considering the adaptability of MGEs as service-oriented architecture or micro-service architecture, addressing the monolithic architecture challenges [New19, MB*21, MP*23]. Previous works analyzed the MGE architecture [Gre18], and the inter-calls between modules, CPU/GPU consumption, and resource requirements [MSK15]. Given strict requirements in XR applications, studies have assessed the computational load and power consumption on client devices [NH*22], the tradeoffs between video quality and latency [MS*21], and offloading rendering tasks [LH*20]. Notable solutions include NVIDIA's CloudXR (<https://www.nvidia.com/en-us/design-visualization/solutions/cloud-xr/>) and the open source ALVR (<https://github.com/alvr-org/ALVR>). Edge-physics frameworks as in [FG*21] propose streaming at the scene-graph level, reporting significantly lower computational overhead, bandwidth, and latency compared to video streaming. [KO23] explores physics computation offloading using the open-source Bevy game engine and Rapier physics engine.

Unprecedented challenges hinder the separation of an MGE's physics engine into a micro-service. Real-time user feedback in MGE-based interactive applications presents obstacles for offloading to edge/cloud resources, risking degraded QoE due to introduced network latency. Synchronizing the simulation state in real-time is complex, especially under actual network conditions. Numerous inter-calls demand high bandwidth, leading to increased costs and potential performance issues. Decoupling an MGE's physics engine is a intricate process heavily dependent on specific engine features, necessitating extensive code modifications that impact other pipeline functionalities.

3. Altering the Game Engine pipeline

The central insight of this proposed system lies in the adoption of a N-1 client-server architecture, specifically implementing one Dissected Physics engine to serve N Graphics applications corresponding to XR users. This architecture facilitates remote physics computations for real-time interactive XR scenes, allowing the offloading of intensive calculations to a single dedicated Physics engine. This engine, containerizable and deployable in cloud resources, is strategically distanced from the HMDs, leading to extended battery life and increased user mobility (identified but not documented in our experiments). The modular pipeline design ensures simplicity for developers, who only need to maintain, support, and extend one module for all Graphics applications and users. The system not only supports real-time deformable surfaces in XR scenes with numerous physics objects but also provides an extendable Physics Engine to accommodate diverse requirements. Despite potential challenges, no alternative solutions have been identified in the literature, underscoring the uniqueness and necessity of the proposed N-1 client-server architecture for addressing the goals and constraints outlined.

The creation of a cohesive and immersive gaming experience, in

a typical MGE pipeline, involves a series of interactions between the graphics rendering and the physics engine. Aiming to influence a 3D object's position and behavior, the physics engine performs CPU computations on 3D object Colliders and RigidBody, utilizing a native solver, responsible to solve motion equations that simulate realistic physics behavior, including collision detection, force impact (gravity, friction, etc.) and joint constraints. The computed simulation results are subsequently combined with rendering parameters, such as GO transform and animation data, to generate the final frame to be displayed.

The proposed dissected MGE pipeline involves two, bidirectionally communicating, units, namely the Graphics Host (or simply *GHost*) and the Physics Server (or simply *PhyS*). The *GHost* unit involves the entire MGE graphics pipeline, including its native local Physics Engine, which remains mostly inactive, as it is only used for the initial connection to the *PhyS* and for UI interactions. The *PhyS* unit involves the MGE physics engine. Main goal of the dissected pipeline, is to allow any interactable GOs on the scene to be fully simulated by the *PhyS* unit instead of the *GHost*'s local physics engine. The two units communicate and get synchronized via network, using primarily UDP connections, through the Photon relay cloud server. These inter-calls transmit either physics simulated GO transformations from the *PhyS*, or user interaction events captured from the *GHost*.

3.1. Splitting a GameObject

A GO is the fundamental building block of a game scene, representing an entity in the game world, serving as a container for components that define its properties, behavior, and appearance, and ultimately contributing to the overall interactivity and visual representation of the game. GOs can have a variety of components attached to them, each serving a specific purpose. A GO can interact with the physics engine through the use of certain components and features that allow the simulation of physical behaviors.

To achieve the physics engine dissection we remove the physics components of the *GHost*'s GOs, but retain the respective physics components within the *PhyS*. In that respect we defined a *Graphics Object*, residing within the *GHost*, as a GO with no physics components or physics-related scripts or data. Additionally, we defined a *Physics Object*, residing within *PhyS*, as a GO with the respective physics components present and active, responsible for storing all physics parameters. It has attached a *RigidBody* script, a *Collider* script, or a combination of the two (see Figure 2).

The *GHost* has no physics components to simulate, and the *PhyS* has no Rendering camera (thus, no Graphics processing takes place). During gameplay, the transformations of the *GHost*'s Graphics Object are synchronized with the respective Physics Object of the *PhyS*. The GO's transformations can either be controlled entirely by the *PhyS*, or by the *GHost*. In the latter case, the Physics simulation continues, and the controlled Physics Object interacts with the rest of the Physics Objects as expected.

The two main units, the *GHost* and the *PhyS*, communicate through Photon Networking, a solution for providing multi-player support in MGE simulations and Games. All GOs assets are known

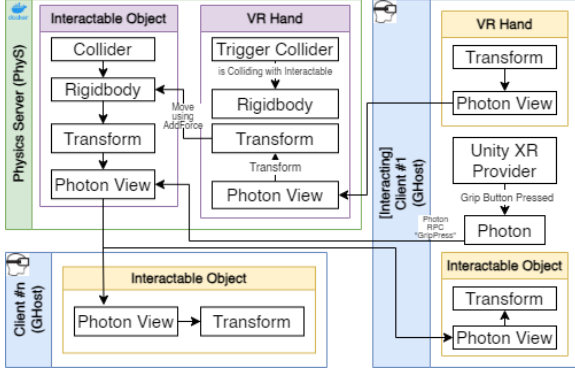


Figure 2: Conceptual architecture of the PhyS dissection.

at build time for both the PhyS and the GHost, allowing the synchronization process across GHost and PhyS units via Photon (see Photon View component in figure 2).

3.2. Multi-user Session initiation & runtime

To allow multi-user game scenes, the first GHost is responsible of deploying the PhyS and initiating a multi-user session. After successful deployment and initiation, the PhyS awaits for other GHost instances to join the multi-user session. The GHosts can use the UI to select the PhyS instance to connect to via a list of available physics servers. After all XR-users have joined the multi-user session, training simulation is started for all connected users. The game logic resides in both the PhyS and GHost so both units are in sync about the current stage of the simulation. Also, any GO that exists in any GHost is dynamically spawned in the PhyS, without any further synchronization. The latter requires that the dynamically spawned GO assets are available in both units.

The GHost unit does not have any control on transforming the Graphics Objects. As such, all Graphics Objects in the participating GHosts are updated with transform data from the corresponding Physics Objects in the PhyS. The only exception to this, is transform computation of the user Hands and Heads. Each GHost is responsible to send the Hands & Camera input transforms to the PhyS. Subsequently, the PhyS applies these transformations to the respective Physics Object, using Physics-based movement, to retain the realistic Physics simulation. The resulting transform (a combination of user input and Physics Simulation) is sent from PhyS back to all participating GHost units, to synchronize the respective Graphics objects and get displayed.

Interaction with objects is achieved by the GHost unit which is responsible of transmitting the user input "Grip Pressed" to the PhyS, every time a user presses their grip button. If the hand object collides an interactable Game Object, the PhyS initiates an interaction by moving the interactable towards the interacting hand, using Physics-based movement. The resulting transforms are then sent from PhyS to all participating GHosts units, to synchronize the respective Graphics objects and get displayed. All game play transformations and multi-user synchronizations are handled by the optimized MAGES networking implementation, which transmits only

Table 1: Average network load

GHost	Upload	Download
Idling player	1.9 KB/s	13.4 KB/s
1 player moving around	3 KB/s	14 KB/s
PhyS	Upload	Download
Idling player	0.4 KB/s	13.3 KB/s
1 player moving around	0.45 KB/s	15.2 KB/s

Table 2: Average rendering times for 300 frames

	Dissected	Monolithic	Improvement
Max	18.84 ms	21.61 ms	12,8% (2.77 ms)
Min	8.46 ms	10.30 ms	17,8% (1.84 ms)
Avg	10.53 ms	12.64 ms	16,7% (2.11 ms)

the necessary data in dynamically-sized network packages, to ensure minimal network usage and optimal QoE.

4. Experimentation

Main objective of the experimentation was to assess the XR-user's QoE and the generated network load for the PhyS using many concurrent XR-users (XR-CCUs > 20). The experiment was conducted using a VR medical training simulation, built with MAGES SDK [ZP*23], based on the Total Knee Arthroplasty operation, which involves several interactable tools that have physics simulations applied to them. It also involves a cutting action with a scalpel [KPA*22] and a bone reaming action with a medical drill.

Due to limited availability of HMDs for experimentation, we developed a desktop application based on the GHost unit to simulate bot-users, achieving experimentation goals for numerous XR-CCUs. The main goal was to create virtual users mimicking human-player movements in multi-user sessions, to simulate network and computational load on the dissected PhyS, similar to sessions with human-players. It interacts with the MAGES UI to join game sessions, then dynamically moves in the scene and perpetually mimics fidgeting movements. The final scene depicts all bot-users seated amphitheatrically (see Figure 3).

After successfully experimenting in a local network setup, we performed a remote experiment with many XR-CCUs (24), assess-



Figure 3: User's View from the experiment with 24 CCUs.

ing the QoE for each one of them, and the network load in real network conditions. A snapshot of this session may be seen in Figure 1. In this setup, we used: a) the PhyS, deployed on cloud resources as docker container on Kernel-based Virtual Machine using QEMU with 12 cores, 3GHz per core, b) 4 XR-CCUs (with Meta Quest HMD) connected through WIFI-internet from various geolocations, c) the Photon cloud Relay Server and d) 20 simulated bot-users connected through internet from a separate machine.

The observed minimal network load (see Table 1) is due to the exploitation of the integrated interpolation engine [KLP21], implemented in MAGES SDK Photon subsystem, allowing fewer animation key-frame network updates (PhyS to GHost syncs), calculating all intermediates. This optimization, empowered with the compression of the broadcasted data saves bandwidth, without compromising the provided frame-rate and QoE. All experiments showcased a good QoE under actual network conditions. Table 2 shows a comparison of the average total rendering time, for the dissected PhyS pipeline compared to the original monolithic HMD application, in actual network conditions, which is improved by 2.11ms. Individual captured metrics showed that GHost physics computations were only improved by 0.1ms, and they are not indicative of the overall improvement, due possible overhead created by Photon relay server.

5. Conclusions and Future work

The design decisions implemented in this system demonstrate a compelling alignment with the stated goals. The strategic offloading of Physics computations from the HMD to the cloud/edge continuum stands out as a pivotal choice, resulting in a remarkable reduction in rendering time. The containerizable architecture further enhances these benefits, making the Physics computations seamlessly deployable in cloud resources. The establishment of a single point of truth for physics computations not only fosters better synchronization in multi-player scenarios but also does so without introducing unnecessary overhead in single-player experiences. The maintenance of the Physics state at the dedicated engine, regardless of users joining, leaving, or participating in the session, ensures the continuity of an always-active session. Additionally, the incorporation of a Geometric algebra interpolator minimizes inter-calls between dissected parts, preserving an equivalent QoE while alleviating network stress. Collectively, these design decisions contribute cohesively to the successful achievement of the stated goals, showcasing a well-thought-out and effective approach to optimizing XR gaming experiences.

The system introduces a transformative capability by demonstrating the feasibility of Physics engine offloading through experimental validation. This capability not only confirms the technical viability of the proposed architecture but also opens up new avenues for enhancing XR experiences. Moreover, the system's ability to provide a commendable QoE for interactive multi-user XR scenes in real network conditions signifies a substantial leap forward. This capability holds profound implications, promising more immersive and collaborative XR applications without compromising performance. The successful execution of offloading computations and ensuring optimal QoE underlines the system's potential

to redefine the standards for multi-user XR engagements, fostering a more inclusive and engaging virtual environment.

In the future, we plan to optimize PhyS by generating smaller dynamically-sized packets for transmission to GHost, to improve further QoE even for degraded network conditions. We also aim to explore integrating Riptide Networking for multiplayer functionality and implementing multi-threaded physics algorithms and GPU compute shaders to leverage edge/cloud resources.

Acknowledgment

This work is supported by the CHARITY project that has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 101016509.

References

- [FG*21] FRISTON S., GRIFFITH E., ET AL.: "quality of service impact on edge physics simulations for vr". *IEEE Transactions on Visualization and Computer Graphics* 27, 5 (2021), 2691–2701. 2
- [Gre18] GREGORY J.: *Game engine architecture*. CRC Press, 2018. 2
- [KLP21] KAMARIANAKIS M., LYDATAKIS N., PAPAGIANNAKIS G.: Never 'drop the ball' in the operating room: an efficient hand-based vr hmd controller interpolation algorithm, for collaborative, networked virtual environments. In *Computer Graphics International Conference* (2021), Springer, pp. 694–704. 4
- [KO23] KURT F. M., OZGOVDE B. A.: Edge computing for computer games by offloading physics computation. *Gazi Univ. Journal of Science Part A: Engineering and Innovation* 10, 3 (2023), 310–326. 2
- [KPA*22] KAMARIANAKIS M., PROTOPSALTIS A., ANGELIS D., TAMIOLAKIS M., PAPAGIANNAKIS G.: Progressive tearing and cutting of soft-bodies in high-performance virtual reality. In *ICAT-EGVE 2022* (2022). ISSN: 1727-530X. 3
- [LH*20] LAI Z., HU Y. C., ET AL.: Furion: Engineering high-quality immersive virtual reality on today's mobile devices. *IEEE Trans. Mob. Comput.* 19, 7 (2020), 1586–1602. 2
- [Liu18] LIU Q.: *Integrating Game Engines into the Mobile Cloud as Micro-services*. PhD thesis, University of Saskatchewan, 2018. 2
- [MB*21] MAKRIAS A., BOUDI A., ET AL.: Cloud for holography and augmented reality. In *2021 IEEE 10th Intl. Conf. on Cloud Networking (CloudNet)* (2021), pp. 118–126. 2
- [MP*23] MAKRIAS A., PSOMAKELIS E., ET AL.: Streamlining xr application deployment with a localized docker registry at the edge. In *Service-Oriented and Cloud Computing* (2023), Springer Nature, pp. 188–202. 2
- [MS*21] MEHRABI A., SIEKKINEN M., ET AL.: Multi-tier cloudvr: Leveraging edge computing in remote rendered virtual reality. *ACM Trans. Multimedia Comput. Commun. Appl.* 17, 2 (2021). 2
- [MSK15] MESSAOUDI F., SIMON G., KSENTINI A.: Dissecting games engines: The case of unity3d. In *NETGAMES* (2015), IEEE, pp. 1–6. 2
- [New19] NEWMAN S.: *Monolith to microservices: evolutionary patterns to transform your monolith*. O'Reilly Media, 2019. 2
- [NH*22] NYAMTIGA B. W., HERMAWAN A. A., ET AL.: Edge-computing-assisted virtual reality computation offloading: An empirical study. *IEEE Access* 10 (2022), 95892–95907. 2
- [PP20] PROTOPSALTIS A., PAPAGIANNAKIS G.: Virtual reality: a model for understanding immersive computing. *Encyclopedia of Computer Graphics and Games*. Springer International Publishing (2020), 1–4. 1
- [ZP*23] ZIKAS P., PROTOPSALTIS A., ET AL.: Mages 4.0: Accelerating the world's transition to vr training and democratizing the authoring of the medical metaverse. *IEEE Computer Graphics and Applications* 43, 2 (2023), 43–56. 3